Analysis of time-series data from a gravitational wave detector using advanced machine learning methods

Christian Gottschlich[1]

[1]University of Bonn

Code: https://github.com/Br1ck11/hfgw-ml-detection

12 Feb 2026, Supervised by Kristof Schmieden

High-frequency gravitational waves represent a largely unexplored observational window where conventional detection analysis techniques are limited by noise and sensitivity constraints. In this report, a hybrid CNN-LSTM classifier network is applied to time-series data to perform binary classification of signal and noise. The model is trained on simulated signals from sub-solar mass primordial black hole (PBH) mergers. Several key mechanisms through which the network learns to distinguish between signal and noise are discovered and explained using explainable AI techniques. This work achieves a false positive rate of fewer than one per year, with the trained network generalizing across signals spanning several orders of magnitude in PBH mass.

1. Introduction

The detection of gravitational waves (GWs) is experimentally challenging, particularly in the high-frequency regime. In this domain, expected signal strains are extremely small, and conventional interferometric techniques rapidly lose sensitivity [1]. Microwave cavity experiments [2] offer a promising alternative for probing high-frequency gravitational waves (HFGWs) through their coupling to the electromagnetic modes; however, the resulting signals are typically deeply buried in noise and subject to instrumental and environmental effects. This work investigates the use of machine-learning–based classification methods to identify HFGW-induced signatures in microwave cavity data from the SUPAX experiment [2]. The HFGWs discussed in this work stem from mergers of sub-solar mass primordial black holes (PBHs) [3] .

2. Methods

Supervised classification is a machine learning (ML) paradigm in which a model is trained to assign input data to one of several predefined classes using examples for which the correct labels are known. During training, the model learns a mapping from the input space to the label space by identifying patterns and structures in the data that are predictive of the assigned classes. Once trained, the classifier can be applied to previously unseen data to quickly infer the likeliest class membership based on the learned representation.

In the context of computer vision, a common architectural choice for supervised classification tasks is the use of Convolutional Neural Networks (ConvNets) as feature extractors [4]. ConvNets incorporate strong inductive biases, most notably locality and translation equivariance; for image and image-like data, these assumptions are generally justified. These properties enable them to efficiently learn hierarchical feature representations, ranging from low-level local patterns to increasingly abstract and semantically meaningful features at deeper CNN layers. As a result, ConvNets have become a standard and highly effective tool for extracting discriminative features from noisy inputs. This makes them a natural candidate to choose for detecting HFGW signals embedded in noise.

While ConvNets are highly effective at extracting local discriminative features from individual data segments, their design and local nature limit their ability to explicitly model long-range temporal dependencies. In many signal detection problems, including the search for HFGWs, the signal's content at any single time step may offer little information. Statistical significance is only achieved when evidence is collected over an extended time interval. To address this, it is natural to combine a ConvNet-based feature extractor with a

temporal integration mechanism that operates on the sequence of extracted features. Such a decomposition separates the learning of local, translation-equivariant representations from the modeling of temporal correlations and evidence accumulation, thereby introducing a physics-informed inductive bias that is better suited to the underlying structure of the problem. This hybrid architecture should, in theory, enable the classifier to integrate information over time while retaining the extractive power of ConvNets.

For the temporal integration of the extracted features, a long-short-term-memory (LSTM) layer [5] in the many-to-one operation mode is used. Figure 1 depicts the exact architecture used in this work. Figure 1 illustrates the architecture of the Convolutional Neural Net (CNN) LSTM classifier used. The complete set of Hyperparameters (HPs) used for training and inference is summarized in Appendix A in tabular form.

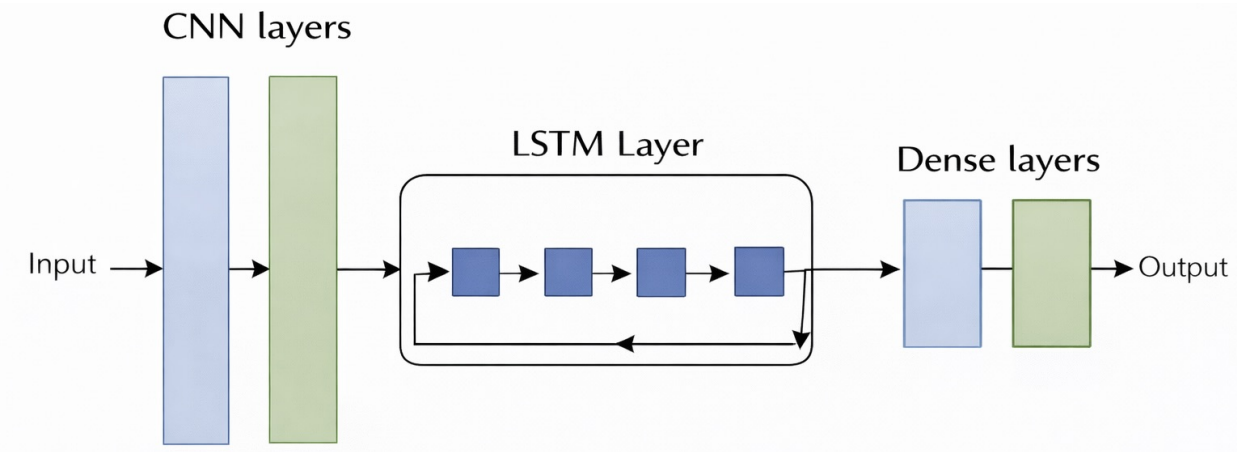

Figure 1: Schematic of the CNN-LSTM Classifier architecture. As an input passes through the network, the first CNN layer extracts local features from the time-series data. A second CNN layer further unifies these features and produces a higher-level representation, which is then passed to an LSTM layer to capture temporal dependencies across the sequence. The output of the LSTM is subsequently processed by a fully connected layer that performs a non-linear combination of the learned temporal features, followed by a linear output head (a D(1) layer) that yields the final classification output score.
The figure was created with GPT 5.2 by OpenAI and partly by [6].

2.1. Data Processing Pipeline

Each input to the model is a preprocessed segment of 1024 data points, referred to as a "window", of time-series data from files of the SUPAX experiment [2]. Each file represents eight seconds of measuring time at a sampling rate of 14 MHz and is saved in `.tiq` format.

The preprocessing works as follows: A single file is loaded and divided into disjoint training, validation, and test segments. Within each segment, a preset number of synthetic signals (see chapter 4.3 of [7] for concrete pictorial examples) are injected at random times, with the start index randomly sampled from a uniform distribution. The signal simulation employed is described in chapter 4 of [7] . It is, however, particularly relevant to mention that the real part of the simulated signal is injected into the I quadrature and the imaginary part into the Q quadrature of the segmented `.tiq` file. For further details on I and Q quadratures, see chapter 2.3 of [7] .

The signal amplitudes are rescaled to pre-specified multiples of the standard deviation $\sigma = 2.68 \times 10^{-5}$ of the training segment's noise. Injection locations are tracked using a boolean mask that flags all data points belonging to a signal. Before dividing the segment into smaller windows that will serve as input to the network, the magnitude $\sqrt{I^2 + Q^2}$ of the I and Q quadratures is computed for each segment to obtain an amplitude time-series. Afterwards, each segment is subdivided into overlapping windows, each of length 1024 data points [1]. A window is then labeled as containing a signal if at least one injected signal sample is present within it; otherwise, it is labeled as noise-only.

[1] Overlapping here refers to using a step size less than the window length when windowing a segment, such that consecutive windows overlap.

2.2. Training Techniques

A common approach taken in two-class classification is to use binary cross entropy (BCE) [8] as the loss function to minimize. This enforces the separation of the two classes: "signal" and "noise", in the case of the detection of HFGW signals. The input for the BCE loss thereby consists of values between 0 and 1. To ensure inputs abide by this restraint, most approaches apply a sigmoid activation function [9] before the loss function is called.

In this work, however, a different approach is taken. While sigmoid activations are standard for probabilistic outputs, their saturation in the tails reduces numerical resolution precisely in the regime relevant for extreme false-positive control. We found this behavior to be detrimental to achieving a target false positive rate (FPR) of fewer than one false positive (FP) per year. As an alternative Ansatz, a linear activation function is used for the final D(1) head, directly outputting the raw logits [2] . Additionally, binary focal cross-entropy (BFL) [10] is used as the loss function instead of BCE. This choice was found to be effective in reducing large noise outliers by placing relative emphasis on these via a weight balancing factor $\alpha = 0.25$ and a focal factor $\gamma = 4$. The focal factor γ increases the relative importance of "hard-to-detect" examples.

For training the CNN-LSTM classifier, as described in Section 2, we used only signals corresponding to a PBH mass of $m_{\mathrm{PBH}} = 10^{-8}\, M_{\odot}$ [3] . We perform the preprocessing and injection of signals as described in Section 2.1, and we list the exact configuration and HPs used for the CNN-LSTM classifier network in Appendix A (Table 1 for pre-training and Table 2 for fine-tuning).

3. Results

It is important to mention that all results are based on the following assumptions

- A sampling rate of 14 MHz is used, as this corresponds to the sampling frequency of the files used. This is relevant to computing the total number of samples per year and the threshold calibration of the model.
- A year is assumed to have $365 \times 24 \times 60 \times 60\,\mathrm{s} = 3.1536 \times 10^{7}\,\mathrm{s}$. This is relevant to computing the total number of samples per year by

$$\text{samples per year} = \text{sampling rate} \times 3.1536 \times 10^{7}\,\mathrm{s} \tag{1}$$

- The model was tested with a window overlap of 96.66%, meaning a rounded step size of 34 was used [4] . This is relevant to estimating the number of windows that the model processes per year and ensures that each signal appears in more than one input window, thereby increasing the number of detection opportunities. The main reason for using a higher overlap than in training was to increase the number of windows for the threshold calibration, which is discussed in Section 3.1 . This overlap was then kept for model testing.

3.1. Threshold calibration

In total, a signal is defined as detected if at least one of its associated windows is correctly classified by the model, i.e. if the model output exceeds a predefined detection threshold. The detection threshold is calibrated to obtain fewer than one FP per year.

Several files (specifically, three were used) are preprocessed as described in Section 2.1, with the important caveat that no signal is injected. The final windows, which consist purely of noise, are then fed into the trained and fine-tuned model. The output values are then obtained from the D(1) layer of the CNN-LSTM network. Figure 4 in Appendix B shows the resulting output distribution.

[2] Logits refer to the generally unbound input values of a sigmoid function.
[3] $M_{\odot} = 1.98 \times 10^{30}$ kg [11] is the solar mass.
[4] Note that a step size of 1024/30 is used, which is rounded to the nearest integer number 34.

Since the available dataset is orders of magnitude smaller than a year's worth of time-series data, the threshold for fewer than one FP per year is found by determining the FPR as a function of the set threshold (2) . The tail, which is defined as the points below or at 1×10^{-4} % of the maximum y-axis value of Figure 5, is then extrapolated using a linear function in log-space, as shown in Figure 5 in Appendix B. The detection threshold is then determined by first computing the number of windows processed per year and setting a target FPR of one FP per year. The log-linear fit is then used to estimate the threshold corresponding to this rate. To obtain fewer than one FP per year, a safety margin of 1×10^{-2} was added to the extrapolated threshold [5]. We find the threshold to be 2.9988.

$$\text{FP per year} = \text{FPR(threshold)} \times \text{Windows per year} \quad \Longleftrightarrow \quad \text{FPR(threshold)} = \frac{\text{FP per year}}{\text{Windows per year}} \tag{2}$$

3.2. Efficiency results

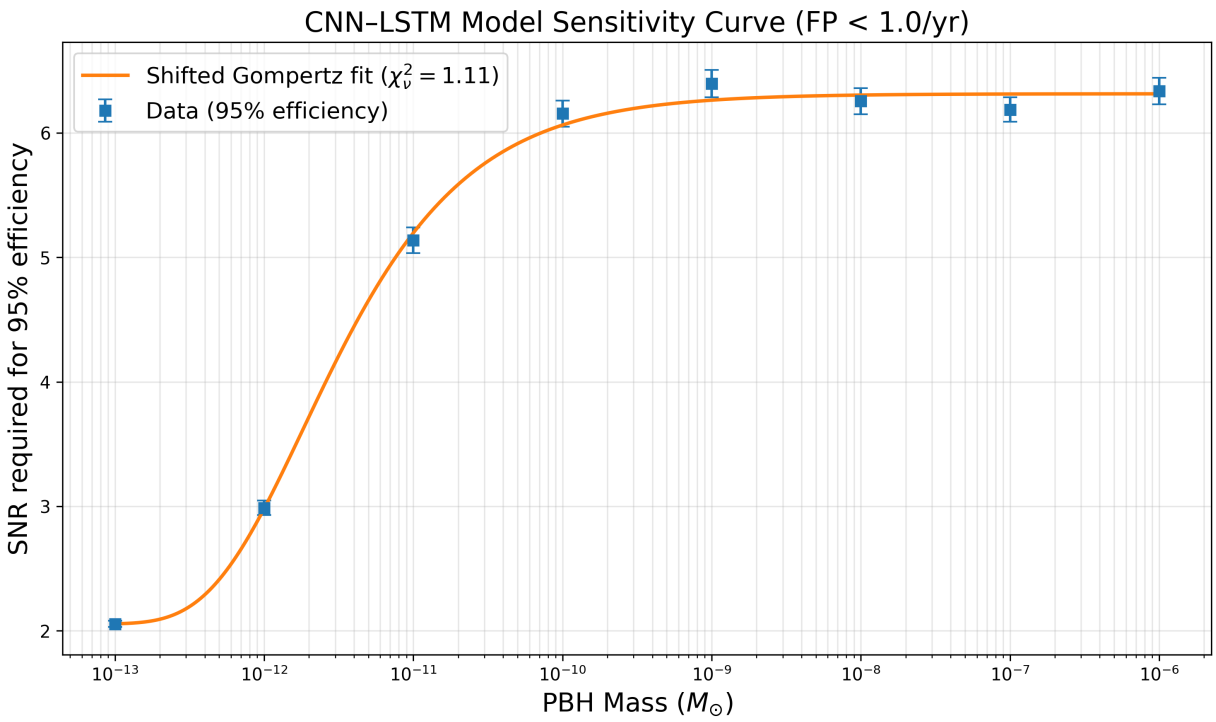

Figure 2: Efficiency curve result to achieve 95% detection efficiency for simulated HFGW signals from merging PBHs. Detection efficiency is defined such that a signal is counted as detected if at least one of its associated windows exceeds a preset threshold. The efficiencies were achieved with a false positive rate (FPR) of fewer than one FP per year.

To evaluate the detection efficiency, signals of a given mass and signal-to-noise ratio (SNR) are injected exactly once into a separate evaluation time-series (distinct from the training and validation data), thus avoiding accidental signal overlap. This procedure is repeated for 500 independent injections to obtain statistically robust efficiency estimates. Repeating the analysis over a range of SNRs and PBH masses yields characteristic S-shaped efficiency curves (see Figure 6 in Appendix C), where the errors in the efficiencies are estimated using Wilson-95 confidence intervals [12]. The basis for this is that, in every run, the model either succeeds in detection or fails, thereby performing a Bernoulli trial. For many independent trials, this

[5] It should be noted that this margin is a heuristic; alternatively, targeting a stricter FPR (e.g., 0.99 per year) would achieve a similar safety margin.

leads to a binomial distribution. The Wilson-95 intervals represent asymmetric intervals that capture the errors for efficiencies near 1 and near 0 more accurately than standard binomial errors.

Subsequently, the obtained efficiency curves are fitted with a sigmoid function (3), from which the SNR corresponding to a detection efficiency of 95% is extracted.

$$\sigma(x) = \frac{1}{1 + e^{-k(x-x_0)}} \tag{3}$$

The uncertainty in this threshold is estimated from the covariance matrix of the fitted parameters k and x_0. Finally, the resulting 95% efficiency points, together with the respective uncertainties, are plotted as a function of the PBH mass for a range of $1 \times 10^{-13}\, M_\odot \leq m_{\mathrm{PBH}} \leq 1 \times 10^{-6}\, M_\odot$. Figure 2 shows the result.

The fitted curve in Figure 2 corresponds to a modified Gompertz function of the form shown in Eq. (4).

$$g(x) = A + (L - A) \cdot e^{-e^{-k \cdot (x - x_0)}} \tag{4}$$

Despite the restricted training set that uses only a PBH mass of $1 \times 10^{-8}\, M_\odot$, the model generalizes beyond the training mass. This can be attributed to the strong similarity of signal morphologies across the considered mass range, which is further explained in chapter 4.3 of [7].

The needed SNR to obtain 95% detection efficiency remains roughly constant for masses $1 \times 10^{-10}\, M_\odot$ and above. This behavior likely arises because the response time of the cavity is larger than or equal to the time a signal is within the resonance band of the cavity for these heavier masses. Lighter masses, in comparison, below $1 \times 10^{-10}\, M_\odot$ are longer within the resonance band and can therefore excite multiple oscillations in the electric field of the cavity. This causes the signal to be longer and easier to detect. For further information, see chapter 4 of [7] .

3.2.1. Mechanism of Detection

For comparatively long-duration signals (relative to PBH masses of $m_{\mathrm{PBH}} \lesssim 1 \times 10^{-11}\, M_\odot$) corresponding to light PBH masses, particularly for masses $m_{\mathrm{PBH}} \lesssim 1 \times 10^{-12}\, M_\odot$, the dominant detection mechanism of the CNN–LSTM classifier appears to be a systematic shift in the mean value of the analyzed time-domain window. In this mass regime, the signal extends over a sufficiently large fraction of the window (or the whole window) such that its contribution leads to a measurable offset relative to the background noise's mean value. This behavior is illustrated in Appendix E in Figure 7, where the injected signal produces a clear change in the window mean that can be robustly distinguished from noise-only realizations.

As the PBH mass increases, the signal duration within the detector's resonance band decreases, and the signal morphology progressively transitions into a short, spike-like structure. This behavior is discussed and figuratively shown in detail in Chapter 4 of [7]. For such heavier masses, the induced mean shift within a window of 1024 samples is no longer sufficiently large to enable reliable detection based on first-order statistics alone. Instead, detection relies on identifying localized features that stand out as abnormalities relative to the stochastic background. Inspection of the CNN's kernel weights 3 and 8 indicates that the CNN learns to respond to local spike-like structures.

Inspecting the kernel weights of the first CNN layer, depicted in Figure 3, we observe several spike-like structures for filters 1, 2, 7, and 8, though less pronounced for filter 7, spanning approximately 20 kernel weights. Additionally, when we inspect the kernel weights of the second CNN layer shown in Figure 8 in Appendix E, the aforementioned CNN layer filters are attributed the most weight by the second layer. This is evident from the consistently higher values of the corresponding input feature maps.

Combining these two facts, along with extensive inspection of the model during detection, strongly suggests that the abnormal localized features, crucial for detection, are spike-like structures with approximate widths of 20 data points. These form the basis for the detection of HFGW signals caused by PBH masses $1 \times 10^{-11}\, M_\odot \lesssim m_{\mathrm{PBH}}$. In comparison, the oscillating kernel patterns observed in filters 3, 4, 5, and 6 (see Figure 3) have been found to play a diminished role in signal detection. This is consistent with the systematically lower average weights assigned to these filters by the second CNN layer (see Figure 8).

However, this type of detection mechanism is intrinsically more challenging, as similar transient features can arise purely by chance in the noise.

This effect is particularly pronounced in the case of the amplitude time-series used because the underlying statistics are described by a heavy-tailed Rayleigh distribution (see Chapter 2.3 of [7] for more information). The heavy-tailed nature of the Rayleigh distribution can cause large spike-like fluctuations to occur with non-negligible probabilities, thereby imitating a signal and potentially fooling the model.

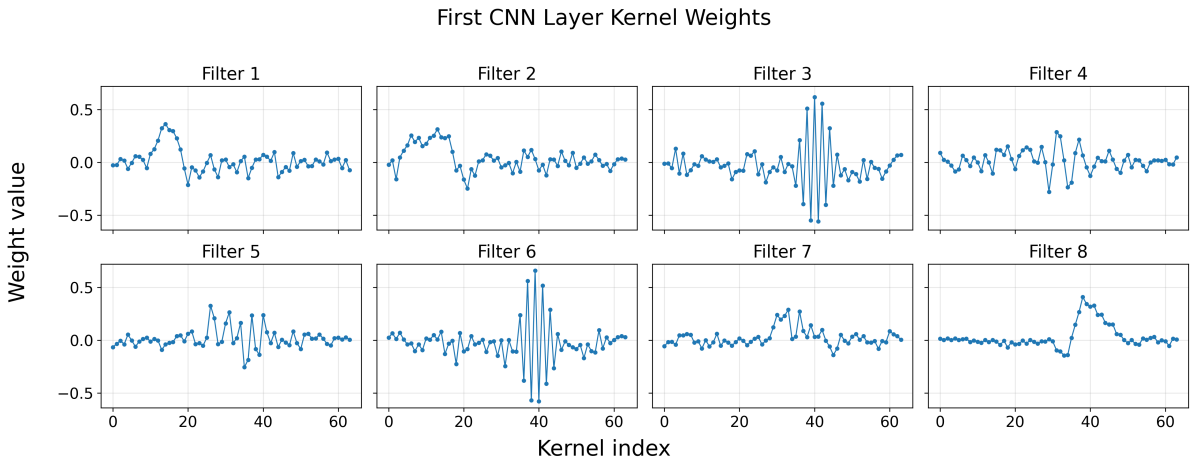

Figure 3: Kernel weights of the first CNN layer. A kernel size of $K = 64$ is used for the first CNN layer's filters, represented on the x-axis. The y-axis represents the learned values for each kernel weight.

In combination, the detection efficiency of the CNN–LSTM classifier can be interpreted as the result of two distinct but mass-dependent mechanisms. For PBH masses below approximately $1 \times 10^{-11}\, M_\odot$, detection is predominantly driven by the classifier's sensitivity to signal-induced mean shifts within the analysis window. For PBH masses $1 \times 10^{-11}\, M_\odot \lesssim m_{\mathrm{PBH}}$, on the other hand, detection increasingly relies on the extraction of short-duration, spike-like structures embedded in the noise, for which the separation between true signals and stochastic fluctuations becomes less pronounced as the SNR shrinks.

4. Conclusion

In this work, we developed a CNN–LSTM classifier (see Figure 1) that operates on raw logits to detect HFGW signals embedded in noise. It was found that for a target detection efficiency of 95% and an explored range of primordial black hole (PBH) masses of $1 \times 10^{-13}\, M_\odot \leq m_{\mathrm{PBH}} \leq 1 \times 10^{-6}\, M_\odot$, the needed SNR as a function of the PBH mass follows a modified Gompertz-function, discussed in Section 3.2 . The two primary mechanisms through which the network separates signal from noise have been found to be

- the mean shift induced in the noise for signals coming from PBH masses below $1 \times 10^{-11}\, M_\odot$ (see Section 3.2) and
- local spike-like structures in the time-series. Here, the CNN stem of the architecture shown in Figure 1 effectively learns to adjust its kernel weights (see Figure 3) into spike-like structures based on the induced signal morphologies.

5. Outlook

Considering the potential problems associated with the heavy-tailed Rayleigh-distribution discussed in Section 3.2.1, a natural choice would be to operate directly on the I and Q quadratures of the `.tiq` files. This is because both quadratures are Gaussian distributed, which reduces the probability of large (meaning above 3σ) outliers. It is worth mentioning that raw I and Q components have been tried in model training, but no additional benefits have been observed; though this is certainly an approach worth considering.

Additionally, it is important to examine the limits of a CNN-based feature extraction stage for time-series data to assess sensible ways to proceed. This can be explored using analytical derivations or by extensively training CNN-based architectures.

Lastly, a Transformer could be explored as an alternative to the LSTM for temporal integration. However, the CNN constitutes the essential feature-extraction stage since it acts directly on the raw time-series, making it the primary choice for further exploration.

6. Acknowledgments

It should be noted that drafting support from Gemini 3 of Google and GPT 5.2 of OpenAI was utilized. In addition, I would like to thank Kristof Schmieden as well as Swarangee Saraf for constructive conversations.

References

[1] Denis Martynov et al. "Exploring the sensitivity of gravitational wave detectors to neutron star physics". In: *arXiv preprint* (2019). arXiv: 1901.03885 [gr-qc].

[2] Kristof Schmieden and Matthias Schott. "The Global Network of Cavities to Search for Gravitational Waves (GravNet): A novel scheme to hunt gravitational waves signatures from the early universe". In: *arXiv preprint arXiv:2308.11497* (2023). arXiv: 2308.11497 [gr-qc]. URL: https://arxiv.org/abs/2308.11497.

[3] Gabriele Franciolini, Anshuman Maharana, and Francesco Muia. "Hunt for Light Primordial Black Hole Dark Matter with Ultra–High–Frequency Gravitational Waves". In: *Physical Review D* 106.10 (2022), p. 103532. DOI: 10.1103/PhysRevD.106.103532. arXiv: 2205.02153 [astro-ph.CO]. URL: https://doi.org/10.48550/arXiv.2205.02153.

[4] Ben Athiwaratkun and Keegan Kang. "Feature Representation in Convolutional Neural Networks". In: *arXiv preprint arXiv:1507.02313* (2015). arXiv: 1507.02313. URL: https://arxiv.org/abs/1507.02313.

[5] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780. URL: http://www.bioinf.jku.at/publications/older/2604.pdf.

[6] Ankur Pandey et al. "A Deep Learning-Based Hybrid CNN-LSTM Model for Location-Aware Web Service Recommendation". In: *Neural Processing Letters* 56.5 (2024), p. 234. DOI: 10.1007/s11063-024-11687-w. URL: https://doi.org/10.1007/s11063-024-11687-w.

[7] Christian Gottschlich. *Differentiating High Frequency Gravitational Waves using Machine Learning*. Tech. rep. University of Bonn, 2025. URL: https://github.com/Br1ck11/hfgw-ml-detection/blob/main/Base_report_differentiating_HFGWs_using_ML.pdf.

[8] TensorFlow Development Team. *tf.keras.losses.BinaryCrossentropy — TensorFlow API Documentation*. Accessed: 2026-01-31. 2025. URL: https://www.tensorflow.org/api_docs/python/tf/keras/losses/BinaryCrossentropy.

[9] TensorFlow Development Team. *tf.keras.activations.sigmoid — TensorFlow API Documentation*. Accessed: 2026-01-31. 2025. URL: https://www.tensorflow.org/api_docs/python/tf/keras/activations/sigmoid.

[10] TensorFlow Development Team. *tf.keras.losses.BinaryFocalCrossentropy — TensorFlow API Documentation*. Accessed: 2026-01-31. 2025. URL: https://www.tensorflow.org/api_docs/python/tf/keras/losses/BinaryFocalCrossentropy.

[11] Andrej Prša et al. "Nominal Values for Selected Solar and Planetary Quantities: IAU 2015 Resolution B3". In: *The Astronomical Journal* 152.2 (2016), p. 41. DOI: 10.3847/0004-6256/152/2/41. URL: https://doi.org/10.3847/0004-6256/152/2/41.

[12] NIST/SEMATECH. *7.2.4.1. Confidence intervals*. Section discusses Wilson/Agresti–Coull intervals for proportions. NIST/SEMATECH e-Handbook of Statistical Methods. URL: https://www.itl.nist.gov/div898/handbook/prc/section2/prc241.htm (visited on 09/10/2025).

A. CNN-LSTM classifier training configurations

A.1. CNN-LSTM Classifier pre-training

The configuration used for the pre-training of the CNN–LSTM classifier is shown. Table 1 lists the network architecture, optimization parameters, data preprocessing settings, and signal injection configuration employed during the initial training stage.

Table 1: Pre-training configuration of the CNN-LSTM classifier model.

pre-training config	value
architecture	CNN–LSTM
LSTM operation mode	many-to-one
activation function	SiLU
training epochs	70
optimizer	AdamW
learning rate	1×10^{-3}
weight decay	1×10^{-2}
CNN filters	[8, 16]
CNN kernel sizes	[64, 16]
CNN strides	[4, 4]
CNN spatial dropout	[0.1, 0.2]
LSTM units	64
LSTM input dropout	0.4
Loss function	BFL
focal α	0.25
focal γ	4
used file	`IQDataFile-2024.04.18.19.19.56.152.tiq`
samples per file	1.12×10^{8}
input shape	1024×1
window size	1024
step size	102 (90% overlap)
train / val / test split	0.8 / 0.1 / 0.1
normalization	Z-score based on train segment stats
PBH mass injected	$10^{-8}\, M_{\odot}$
SNR range	[0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0, 6.5, 7.0, 7.5, 8.0, 9.0]
Number of injections (train / val / test)	15000 / 2000 / 2000
batch size	1024
dataset shuffling	enabled

A.2. CNN-LSTM Classifier fine-tuning

After pre-training, the classifier is fine-tuned using a reduced learning rate and batch size as well as fixed normalization statistics. The corresponding configuration is summarized in Table 2.

Table 2: Fine-tuning configuration of the CNN–LSTM classifier model. The pre-trained network is further optimized using fixed normalization statistics and a reduced learning rate.

fine-tuning config	value
loaded model	pretrained CNN–LSTM
input shape	1024×1
window size	1024
step size	102 (90% overlap)
train / val / test split	0.8 / 0.1 / 0.1
normalization	Z-score (fixed statistics)
global mean	5.13×10^{-5}
global std	2.68×10^{-5}
PBH mass injected	$10^{-8}\, M_{\odot}$
SNR range	[0.5, 1.0, 1.25, 1.5, 1.75, 2.0, 2.25, 2.5, 2.75, 3.0, 3.25, 3.5, 3.75, 4.0, 4.25, 4.5]
injections (train / val / test)	15000 / 2000 / 2000
optimizer	AdamW
fine-tuning learning rate	5×10^{-5}
weight decay	1×10^{-8}
batch size	256
dataset shuffling	enabled
loss function	BFL
focal α	0.25
focal γ	4.0

B. Threshold Calibration

This section illustrates the procedure used to calibrate the detection threshold corresponding to fewer than one FP per year. The calibration is based exclusively on noise-only data and relies on extrapolating the tail of the FPR distribution with a linear function in log-space, corresponding to an exponential suppression of extreme events. Due to the lack of 1 year's worth of data, the threshold found of 2.9988 can lead to underestimation or overestimation of the efficiencies.

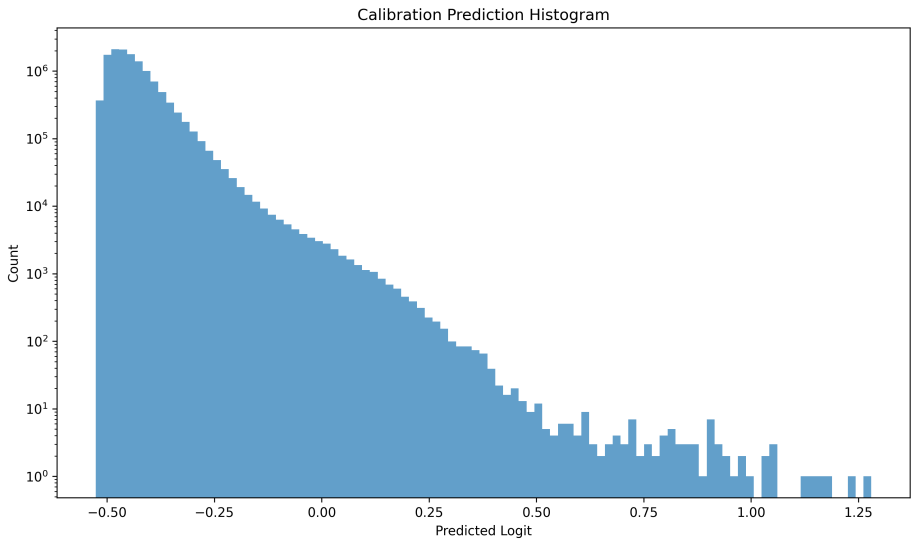

Figure 4: Distribution of the model output logits obtained from noise-only windows during threshold calibration. This distribution forms the basis for estimating the FPR as a function of the detection threshold.

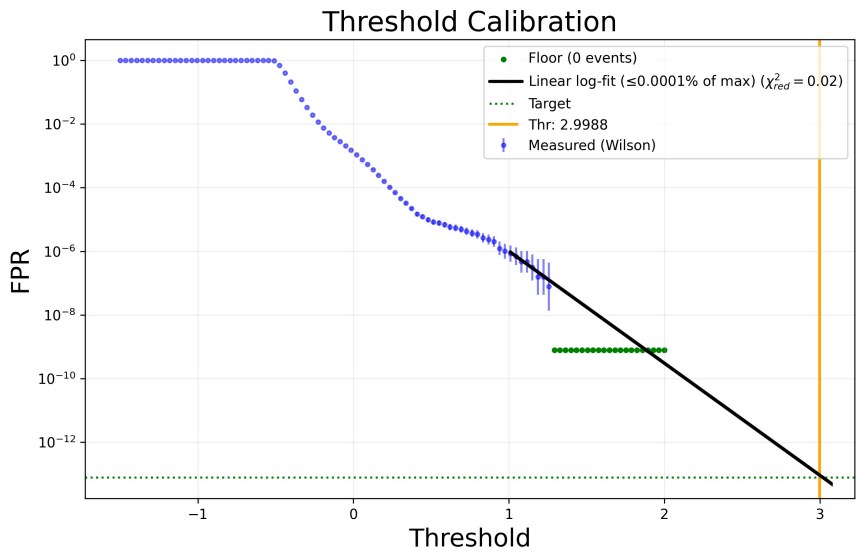

Figure 5: FPR as a function of the detection threshold, shown on a logarithmic y-scale. The tail of the distribution is extrapolated using a linear fit in log-space to estimate the threshold corresponding to one false positive per year. The green points correspond to 0 FPs and are clipped to a finite value because of the logarithmic scaling of the y-axis

C. Efficiency curve

In Figure 6, we see two example efficiency curves that were used for the creation of Figure 2. Both follow an S-shaped curve, whereby the curve shown in the bottom panel of Figure 6 exhibits a steeper drop-off than the one in the top panel. In addition, the top panel's curve is shifted to higher SNR values compared to the curve in the bottom panel.

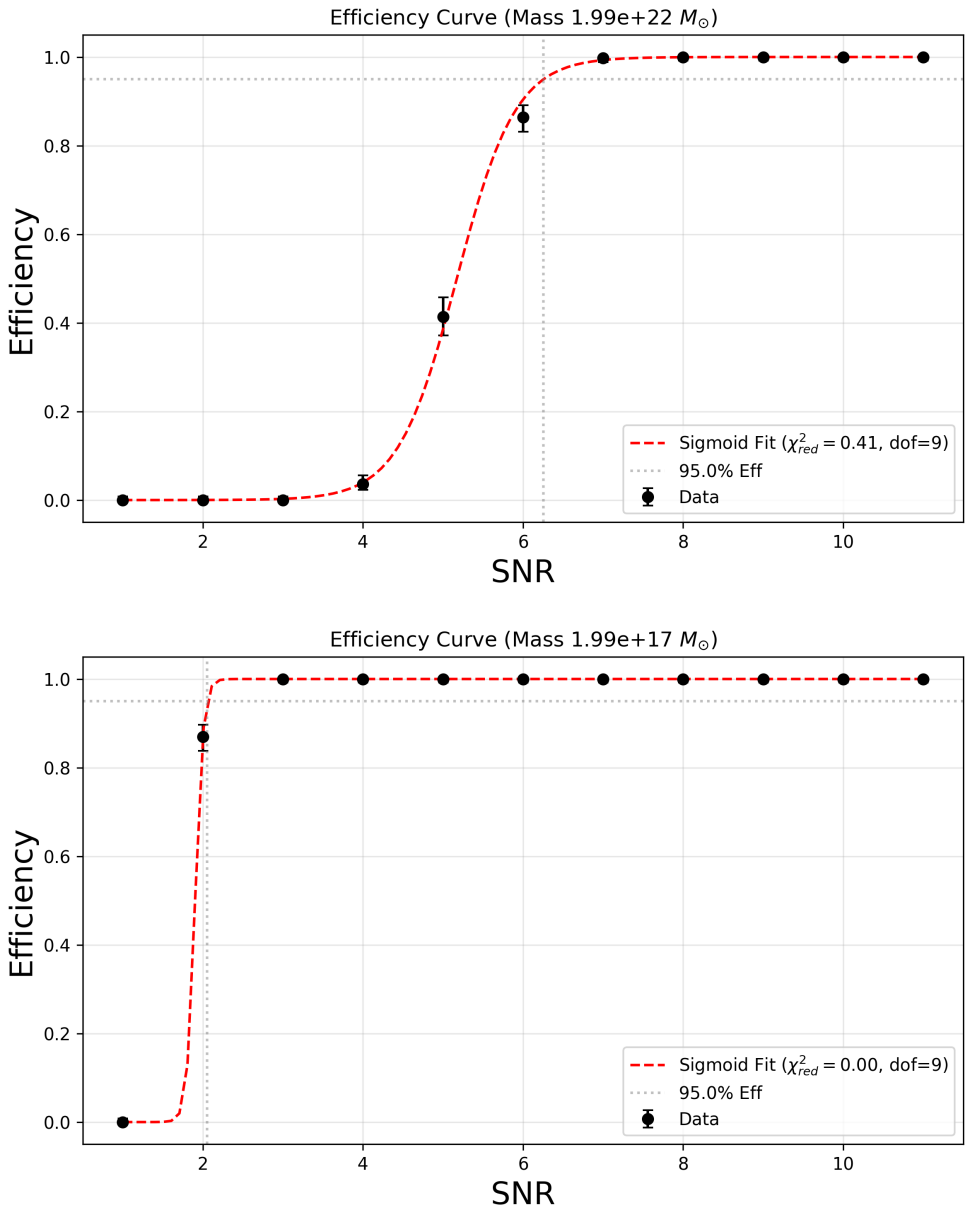

Figure 6: Detection efficiency curves for simulated HFGW signals. The top panel shows the result for signals coming from a mass of $m_{\rm PBH} = 1 \times 10^{-8}\, M_\odot$, the bottom for $m_{\rm PBH} = 1 \times 10^{-13}\, M_\odot$. Error bars are Wilson score intervals at 95% confidence and the dashed red fitting curve corresponds to a sigmoid function with steepness parameter k, controlling the slope of the drop-off, and x-axis shift x_0. Each curve is based on the iterative injection and testing of simulated HFGW signals explained in Section 3.2. The abbreviation dof stands for degrees of freedom and is defined as subtracting the number of fit parameters from the total number of data points.

D. Example Signal and Noise Windows

In Figure 7, we see two examples of an injected signal into noise, originating from a PBH mass of $m_{\mathrm{PBH}} = 10^{-13}\,M_{\odot}$ at an SNR of 2. Figure 7 illustrates the effect on the mean value of a window induced by such signals. The mean value of the upper panel of Figure 7 is thereby an order of magnitude larger than the one of the bottom panel.

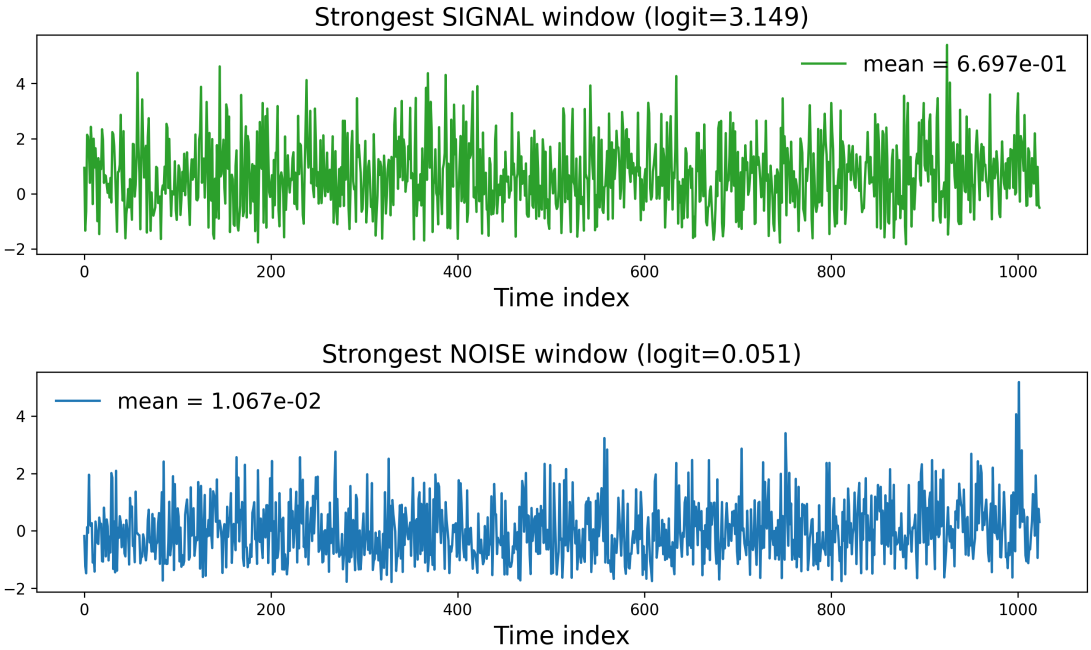

Figure 7: Example time-domain windows for an injected PBH signal with mass $m_{\mathrm{PBH}} = 10^{-13}\,M_{\odot}$ and SNR = 2 (top panel) and a noise-only window (bottom panel). The injected signal spans 32768 samples before dropping below 1×10^{-3} of its peak amplitude[*a], which means a continuous non-zero offset is added across the entire 1024-sample analysis window. This causes a systematic shift in the window's mean value compared to background noise, a deviation that the CNN–LSTM classifier explicitly exploits for detection.

[*a] It should be noted that a signal itself is cropped to only contain amplitude values of 1×10^{-3} or above. For further information it is referred to Chapter 4.3 of [7] .

E. Kernel weights heatmap of 2nd CNN layer

In Figure 8 we see the kernel weight distributions for the second CNN layer's 16 filters. Each 16×8 heatmap displays how the layer weights the 8 input features from the first CNN layer. The color scale spans from -0.72 (purple) to 0.72 (yellow). Filters 1 and 3 exhibit the strongest weight magnitudes, while most filters show moderate, relatively uniform weight distributions centered around zero (teal regions).

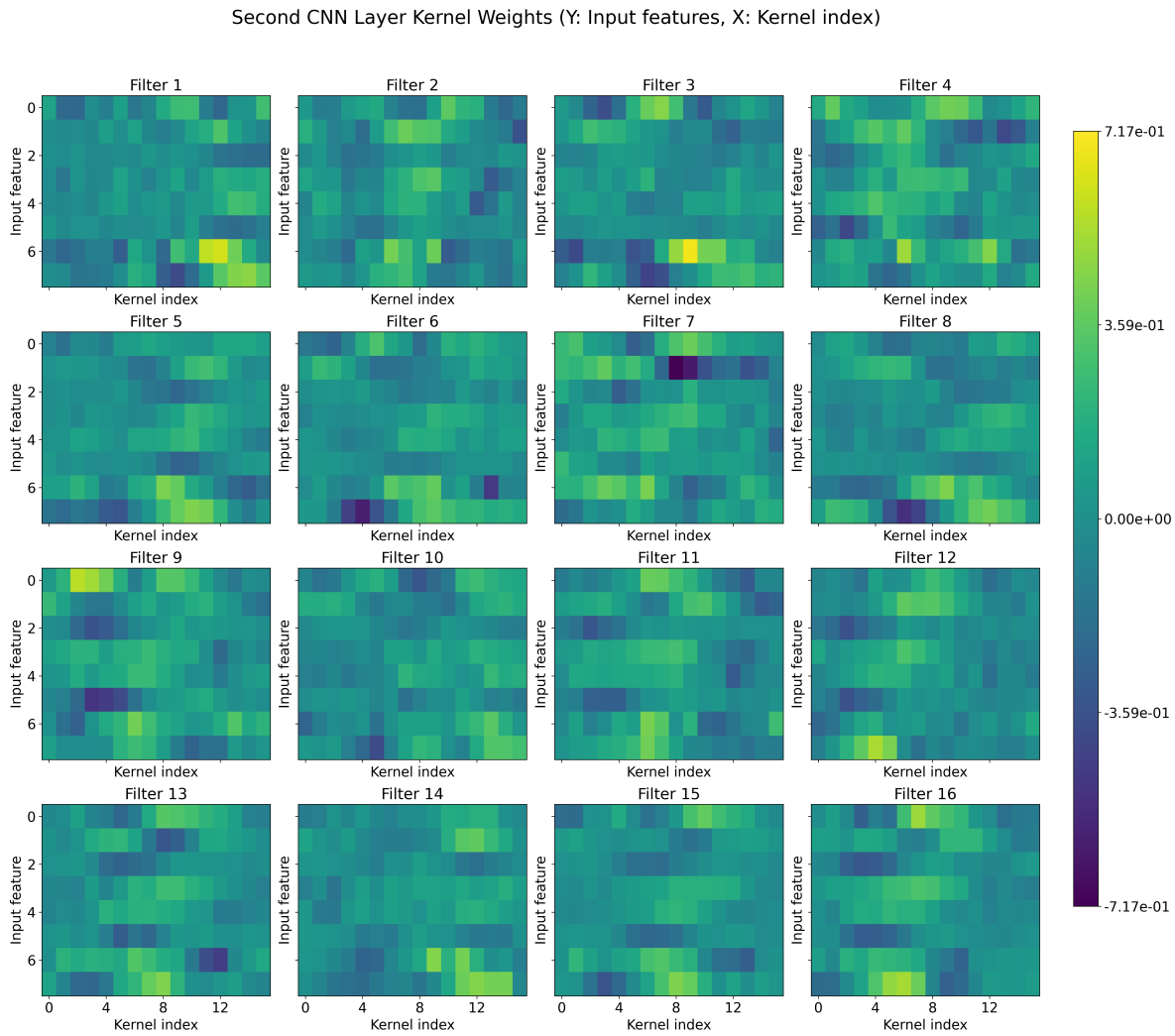

Figure 8: Kernel weight heatmap of the second CNN layer. A kernel size of $K = 16$ is used for the second CNN layer filters, represented on the x-axis. The 8 input features on the y-axis represent the 8 feature maps coming from the filter of the first CNN layer. Each heatmaps therefore shows a patch of 16×8 weights per filter.